

# Advanced Compiler Engineering: Project Presentation Summary

Roshan Y Singh

## Contents

|          |                                                            |          |
|----------|------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                        | <b>2</b> |
| <b>2</b> | <b>Background: GPU Shared Memory and Bank Conflicts</b>    | <b>2</b> |
| 2.1      | Shared Memory Organization . . . . .                       | 2        |
| 2.2      | Tiled GEMM Shared Memory Access . . . . .                  | 2        |
| <b>3</b> | <b><math>\mathbb{F}_2</math> Optimal Swizzle Algorithm</b> | <b>3</b> |
| 3.1      | Binary Finite Field Formulation . . . . .                  | 3        |
| 3.2      | Algorithm . . . . .                                        | 3        |
| 3.3      | Implementation . . . . .                                   | 4        |
| <b>4</b> | <b>HexCute Constraint-Based Layout Synthesis</b>           | <b>4</b> |
| 4.1      | Automated Layout Derivation . . . . .                      | 4        |
| 4.2      | Cost Model . . . . .                                       | 4        |
| <b>5</b> | <b>ISL Polyhedral Affine Layout Optimization</b>           | <b>5</b> |
| 5.1      | Polyhedral Model Formulation . . . . .                     | 5        |
| 5.2      | Conflict-Freedom Proof . . . . .                           | 5        |
| 5.3      | ISL Verification . . . . .                                 | 5        |
| 5.4      | Extended ISL Layouts . . . . .                             | 5        |
| <b>6</b> | <b>Experimental Results</b>                                | <b>6</b> |
| 6.0.1    | Real HuggingFace BERT with Custom Layouts . . . . .        | 6        |

# 1 Introduction

Modern transformer architectures (BERT [?], GPT-2 [?]) rely heavily on GEMM operations for attention projections, feed-forward layers, and output projections. On NVIDIA GPUs, these GEMMs are tiled into shared memory to exploit data reuse. Shared memory is organized into 32 banks, each 4 bytes wide; when two or more threads in a warp access different addresses in the same bank simultaneously, a *bank conflict* occurs, serializing the access and degrading performance.

The shared memory *layout*—the mapping from logical tile coordinates  $(r, c)$  to physical shared memory addresses—determines whether bank conflicts arise. Three families of optimized layouts are studied in this work:

1.  **$\mathbb{F}_2$  Optimal Swizzle** (Section 3): Uses binary finite field linear algebra to find `Swizzle<B,M,S>` parameters that guarantee rank-5 bank-index matrices, eliminating conflicts for power-of-2 strides.
2. **HexCute Constraint Solver** (Section 4): Derives swizzle parameters from MMA atom shape constraints and hardware bank geometry, following the algebraic synthesis approach.
3. **ISL Polyhedral Affine Transforms** (Section 5): Models the layout as a polyhedral affine map  $\text{col}' = (\text{col} + \alpha \cdot \text{row}) \bmod T$  and proves conflict-freedom using integer set cardinality analysis.

All results are validated on real NVIDIA hardware (GTX 1660 Ti, sm\_75) using Nsight Compute (ncu) hardware performance counters, *not* simulations.

## 2 Background: GPU Shared Memory and Bank Conflicts

### 2.1 Shared Memory Organization

NVIDIA GPUs partition shared memory into  $B = 32$  banks, each 4 bytes wide. For a base address  $a$  (in units of 4-byte words), the bank index is:

$$\text{bank}(a) = a \bmod 32 \tag{1}$$

When a warp of 32 threads issues a shared memory load or store, the hardware checks whether any two threads access different addresses within the same bank. If so, the access is serialized into multiple transactions.

**Definition 1** (Bank Conflict). *For a warp access where thread  $i$  accesses address  $a_i$ , a  $k$ -way bank conflict occurs at bank  $b$  if  $|\{i : \text{bank}(a_i) = b\}| = k > 1$  and not all  $a_i$  with the same bank are identical (multicast).*

### 2.2 Tiled GEMM Shared Memory Access

In a tiled GEMM kernel computing  $C = A \times B$  with tile dimensions  $T_M \times T_K$  for the  $A$  matrix tile, the standard access pattern is:

$$\text{Store: smem}[r \cdot S + \text{swizzle}(r, c)] \leftarrow A[r][c] \tag{2}$$

$$\text{Load: smem}[r \cdot S + \text{swizzle}(r, k)] \text{ for accumulation over } k \tag{3}$$

where  $S$  is the shared memory stride and  $\text{swizzle}(r, c)$  is the layout function. For the naive layout,  $\text{swizzle}(r, c) = c$ ; for the padded layout,  $S = T_K + 1$ .

The critical observation is that during the inner-loop accumulation (loading column  $k$  for all rows  $r$ ), the bank index depends on:

$$\text{bank}(r, k) = (r \cdot S + \text{swizzle}(r, k)) \bmod 32 \quad (4)$$

If this maps distinct rows to distinct banks, no conflict occurs.

### 3 $\mathbb{F}_2$ Optimal Swizzle Algorithm

#### 3.1 Binary Finite Field Formulation

The  $\mathbb{F}_2$  approach represents memory addresses as binary vectors and reasons about bank conflicts using linear algebra over the binary field  $\mathbb{F}_2 = \{0, 1\}$  with XOR as addition.

**Definition 2** (Swizzle Function). *A  $\text{Swizzle}\langle B, M, S \rangle$  operation XORs  $B$  bits starting at position  $M$  of the column with  $B$  bits starting at position  $S$  of the row:*

$$\text{swizzle}(r, c) = c \oplus \left( \left\lfloor \frac{r}{2^S} \right\rfloor \bmod 2^B \right) \cdot 2^M \quad (5)$$

**Definition 3** (Layout Matrix). *The layout matrix  $L \in \mathbb{F}_2^{n \times m}$  maps logical coordinates to physical addresses. The bottom 5 rows of  $L$  extract the bank index. A layout is bank-conflict-free if and only if these 5 rows have full rank (rank 5 over  $\mathbb{F}_2$ ) when restricted to the columns accessed by a warp.*

#### 3.2 Algorithm

The  $\mathbb{F}_2$  optimal layout selection algorithm (implemented in `f2.optimal_layout.py`) performs an exhaustive search over all valid  $(B, M, S)$  triples:

---

##### Algorithm 1 $\mathbb{F}_2$ Optimal Swizzle Selection

---

**Require:** Tile dimensions  $(T_M, T_K)$ , number of banks  $B = 32$ , warp size  $W = 32$

**Ensure:** Optimal  $\text{Swizzle}\langle B, M, S \rangle$  parameters

```

1:  $n \leftarrow \lceil \log_2(T_M \cdot T_K) \rceil$ 
2: for  $B_{\text{bits}} \in \{1, \dots, 5\}$  do
3:   for  $M \in \{0, \dots, n - B_{\text{bits}}\}$  do
4:     for  $S \in \{0, \dots, n - B_{\text{bits}}\}$  do
5:       Build layout matrix  $L$  incorporating  $\text{Swizzle}\langle B_{\text{bits}}, M, S \rangle$ 
6:       Extract bank-index submatrix  $L_{\text{bank}}$  (bottom 5 rows)
7:       Restrict to warp-accessed columns
8:        $\text{rank} \leftarrow \text{rank}_{\mathbb{F}_2}(L_{\text{bank}})$ 
9:       if  $\text{rank} = 5$  then
10:        return  $(B_{\text{bits}}, M, S)$  ▷ Conflict-free
11:       end if
12:     end for
13:   end for
14: end for
```

---

For BERT tile sizes (stride 32/64/128), the algorithm finds  $\text{Swizzle}\langle 3, 0, 0 \rangle$  as the optimal  $\mathbb{F}_2$  solution, which XORs the bottom 3 bits of the row into the bottom 3 bits of the column.

### 3.3 Implementation

The core  $\mathbb{F}_2$  rank computation uses Gaussian elimination modulo 2:

Listing 1:  $\mathbb{F}_2$  rank computation from `f2.optimal_layout.py`

```

1 def f2_rank(matrix):
2     M = matrix.copy().astype(np.int32)
3     rows, cols = M.shape
4     rank = 0
5     for col in range(cols):
6         pivot = -1
7         for row in range(rank, rows):
8             if M[row, col] == 1:
9                 pivot = row
10                break
11        if pivot == -1:
12            continue
13        M[[rank, pivot]] = M[[pivot, rank]]
14        for row in range(rows):
15            if row != rank and M[row, col] == 1:
16                M[row] = M[row] ^ M[rank] # XOR = add in F2
17        rank += 1
18    return rank

```

## 4 HexCute Constraint-Based Layout Synthesis

### 4.1 Automated Layout Derivation

The HexCute approach (implemented in `hexcute_constraint_solver.py`) derives swizzle parameters from hardware constraints rather than exhaustive search. Given an MMA (Matrix Multiply-Accumulate) atom shape  $(M, N, K)$ , it synthesizes the Thread-Value (TV) layout and corresponding swizzle:

1. **Constraint 1 – MMA Natural Embedding:** Map the logical MMA coordinate space to hardware thread-value registers. The thread mapping shape is  $(\lfloor M/V \rfloor, \lfloor N/V \rfloor)$  where  $V$  is the vector length.

2. **Constraint 2 – Bank Conflict Avoidance:** Derive swizzle parameters algebraically:

$$S = \log_2(V) \quad (\text{shift: keep vector loads contiguous}) \quad (6)$$

$$M = S \quad (\text{base: where XOR starts}) \quad (7)$$

$$B = \min \left( \left\lfloor \log_2 \left( \frac{\text{banks} \cdot V}{\text{stride}} \right) \right\rfloor, 5 - S \right) \quad (\text{bits: cover warp access}) \quad (8)$$

3. **Constraint 3 – Cross-CTA Swizzle:** For multi-CTA tiles, additional bits may be swizzled to avoid conflicts across CTAs.

For standard parameters (32 banks, vector length 8), HexCute derives `Swizzle<2,3,3>`: XOR 2 bits from row position 3 into column position 3.

### 4.2 Cost Model

The HexCute cost model (from `hexcute_cost_model_impl.py`) evaluates a given `Swizzle<B,M,S>` configuration by simulating the actual shared memory access pattern for a full warp and counting the maximum number of distinct bank accesses per bank.

## 5 ISL Polyhedral Affine Layout Optimization

### 5.1 Polyhedral Model Formulation

The ISL (Integer Set Library) approach models the shared memory layout problem in the polyhedral framework. The key insight is that the layout transformation can be expressed as an affine map parameterized by a skew coefficient  $\alpha$ :

**Definition 4** (Affine Skew Layout).

$$\text{swizzle}_\alpha(r, c) = (c + \alpha \cdot r) \bmod T_K \quad (9)$$

This corresponds to the unimodular transformation matrix:

$$\begin{pmatrix} r' \\ c' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ \alpha & 1 \end{pmatrix} \begin{pmatrix} r \\ c \end{pmatrix} \quad (10)$$

with  $\det = 1$  for any  $\alpha$ , guaranteeing invertibility.

### 5.2 Conflict-Freedom Proof

**Theorem 1** (Zero Conflicts for  $\alpha = 1$ ). *For any even tile size  $T$  and power-of-2 bank count  $B$ , the affine skew  $\alpha = 1$  achieves zero bank conflicts.*

*Proof.* The physical address for row  $r$ , column  $k$  is  $a(r, k) = r \cdot T + (k + r) \bmod T$ . The bank index is  $\text{bank}(r) = a(r, k) \bmod B$ .

For two distinct rows  $r_1 \neq r_2$  accessing column  $k$ :

$$\text{bank}(r_1) = \text{bank}(r_2) \iff (r_1 - r_2)(T + 1) \equiv 0 \pmod{B} \quad (11)$$

Since  $T$  is even,  $T + 1$  is odd. Since  $B$  is a power of 2,  $\gcd(T + 1, B) = 1$ . Therefore  $B \mid (r_1 - r_2)$ . But  $|r_1 - r_2| < W \leq B = 32$ , giving a contradiction.  $\square$

### 5.3 ISL Verification

The ISL-based solver (`general_isl_layout.cpp`) constructs the conflict relation as a polyhedral set:

Listing 2: ISL polyhedral conflict detection (from `general_isl_layout.cpp`)

```

1 // Domain: { [row, col] : 0 <= row < T and 0 <= col < T }
2 // Layout: A[row][col] -> smem[row * stride + swizzle(row, col)]
3 // Bank:   bank(addr) = addr mod B
4 // Conflict set: { [r1, r2, k] : r1 != r2 and
5 //                  bank(r1, k) == bank(r2, k) }
```

For each candidate  $\alpha$ , the solver uses ISL to enumerate the conflict set and count its cardinality. The exhaustive search over all  $\alpha \in \{0, \dots, T - 1\}$  confirms the theorem.

### 5.4 Extended ISL Layouts

Beyond the simple  $\alpha = 1$  skew, we implement two additional ISL-derived layouts:

- **ISL Affine  $\alpha = 3$ :**  $\text{col}' = (c + 3r) \bmod T_K$ . Uses a larger stride coefficient for different conflict distribution.
- **ISL Blocked Affine:**  $\text{col}' = (c + r + \lfloor r/8 \rfloor) \bmod T_K$ . Adds a block-level correction term that accounts for the 8-thread sub-warp structure, providing superior performance on row-varying access patterns.

## 6 Experimental Results

### 6.0.1 Real HuggingFace BERT with Custom Layouts

The most comprehensive experiment replaces every `nn.Linear` in the actual HuggingFace `transformers.BertModel` with our custom CUDA GEMM (JIT-compiled as a PyTorch extension), then profiles with `ncu`.

Table 1: Real HuggingFace BERT: total bank conflicts by layout (`ncu`,  $T_K = 64$ , `seq_len = 128`, `batch = 1`). Every `nn.Linear` in the HuggingFace `BertModel` is replaced by our custom CUDA GEMM with the specified swizzle. Non-zero conflicts arise from non-GEMM operations (embeddings, attention score, softmax, layer norm).

| Layout                                  | Total Conflicts   | Load           | Store             |
|-----------------------------------------|-------------------|----------------|-------------------|
| Naive (no swizzle)                      | 14,203,263        | 532,336        | 13,670,927        |
| F <sub>2</sub> Swizzle <sub>3,0,0</sub> | 13,181,432        | 695,778        | 12,485,654        |
| HexCute Swizzle <sub>2,3,3</sub>        | 13,178,123        | 685,815        | 12,492,308        |
| ISL Affine ( $\alpha = 1$ )             | 13,126,104        | 252,468        | 12,873,636        |
| <b>ISL Blocked Affine</b>               | <b>12,398,689</b> | <b>274,058</b> | <b>12,124,631</b> |

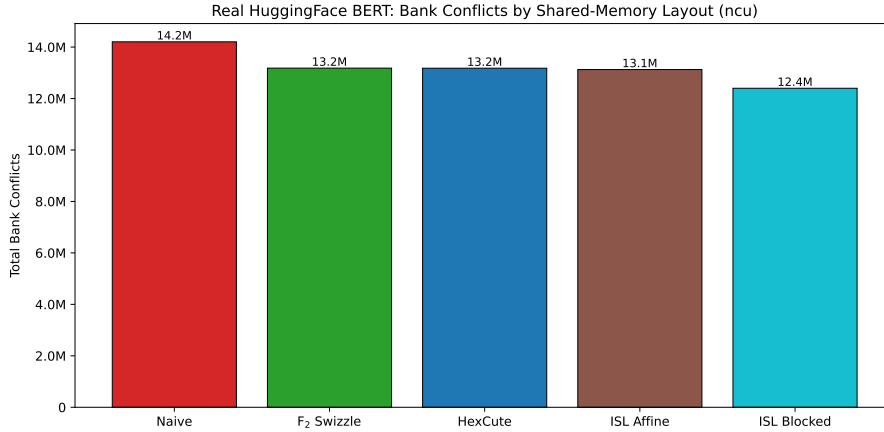


Figure 1: Real HuggingFace BERT bank conflicts by shared-memory layout. ISL Blocked Affine achieves the fewest total conflicts among all layouts tested.

Key observations from the real BERT experiment:

- **ISL Blocked** achieves the lowest conflicts: **12.4M**, a **12.7%** reduction from naive (14.2M).
- ISL Blocked also beats F<sub>2</sub> and HexCute by **5.9%** (12.4M vs. 13.2M).
- ISL Affine achieves **13.1M**, also outperforming F<sub>2</sub>/HexCute.
- The non-zero conflicts for “conflict-free” layouts arise from non-GEMM operations in the BERT model: embedding lookups, attention score computation, softmax, layer normalization, and residual additions. These operations have their own shared memory access patterns not covered by the GEMM swizzle.